

O USO DE LOGS NO ENSINO DE PROGRAMAÇÃO

Universidade Tecnológica Federal do Paraná

Gilmar Gomes do Nascimento

*Universidade Tecnológica Federal do Paraná
Programa de Pós-Graduação em Computação Aplicada*

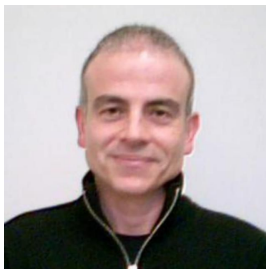
12 de dezembro de 2025

Programação

- 1 Orientadores
- 2 Introdução
 - Objetivos
 - Objetivos
- 3 Metodologia
- 4 Revisão Sistemática da Literatura
- 5 Questão de Pesquisa 1
- 6 Questão de Pesquisa 2
- 7 Survey Exploratório
- 8 Design Science Research
- 9 Algoritmo - desenvolvimento e rastros
- 10 Cronograma
- 11 Artigo - Evento
- 12 Referências
- 13 Agradecimentos

Orientadores

Orientadores



Laudelino Cordeiro Bastos



Maria Claudia Figueireido Pereira Emer



Agradecimentos



Adolfo Gustavo Serra Seca Neto



Introdução

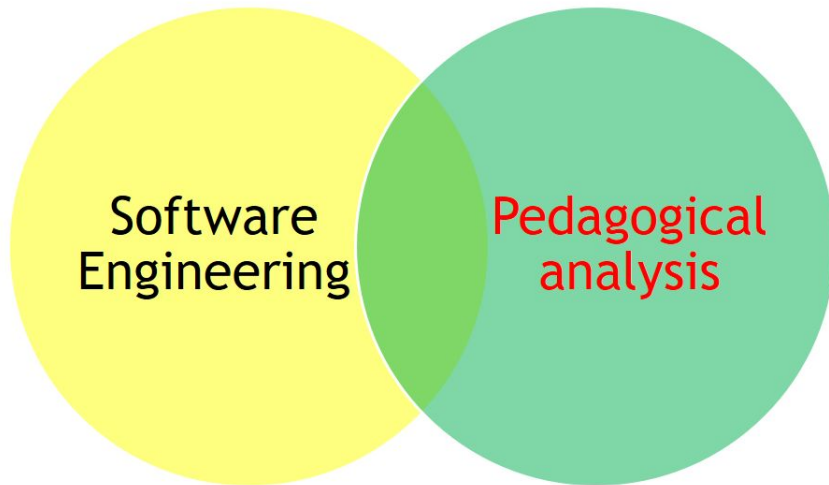
Práxis

O ensino de programação é uma atividade multidisciplinar que demanda uma práxis pedagógica apurada – ou seja, uma constante reflexão que integre teoria e prática em sala de aula. Uma estratégia comum é utilizar exemplos do cotidiano para estimular a resolução de problemas por meio de algoritmos. Contudo, avaliar a qualidade desses algoritmos exige ir além da verificação funcional.

Dificuldades no aprendizado

Ainda segundo (Raabe; Silva, 2005), as dificuldades no aprendizado de programação podem ser categorizadas em três grupos distintos, que vão além da simples aptidão individual: Problemas de natureza didática (relacionados aos métodos de ensino), cognitiva (relacionados aos desafios de raciocínio lógico e abstração) e afetiva (relacionados à motivação, ansiedade e confiança do aluno). Essa categorização sistêmica ajuda a entender que o desafio requer iniciativas diferentes.

Diagrama de Venn: Integração de Disciplinas



Logging: Conceitos e aplicações na Engenharia de Software

As práticas de registro de rastros em sistemas computacionais abrangem diversos conceitos. Como exemplo, a taxonomia de (*Gu et al., 2022*) define os seguintes elementos fundamentais:

- Log statement
- Log level
- Log placement
- Log message
- Log content
- Logging
- Log
- Log location
- Logging practice

A importância dos logs

Segundo (Rice; Borgman, 1983), o monitoramento de sistemas pode ser entendido como um registro automático de transações. A importância dos *logs* é vasta, sendo utilizados em tarefas como detecção de anomalias, depuração, diagnóstico de desempenho e modelagem de comportamento de sistemas (Fu et al., 2014). Estudos como o de (Yang et al., 2021) confirmam que os desenvolvedores os utilizam primariamente para análise de problemas, buscando informações como a propagação de erros, *timestamps* e o *dataflow* entre componentes.

Análise de logs como Ferramenta Educacional

Tabela: Comparação entre abordagens de avaliação em programação

Avaliação Tradicional	Avaliação com Análise de Logs
Foca no produto final (código)	Foca no processo de desenvolvimento
Analisa o que foi produzido	Analisa como foi produzido
Avaliação predominantemente quantitativa	Inclui dimensões qualitativas do aprendizado
Visão estática do conhecimento	Visão dinâmica da curva de aprendizagem
Foco no indivíduo	Permite análise individual e em grupo

Objetivo geral

O objetivo deste trabalho é desenvolver e implementar um plugin para coleta e análise de logs durante o processo de ensino-aprendizagem de programação, com o intuito de fornecer métricas que auxiliem na identificação de dificuldades dos alunos, na personalização do ensino e no aprimoramento das práticas pedagógicas

Objetivos específicos

- 1 O que pretende:** Investigar o processo de aprendizagem de programação por meio da análise de logs acadêmicos, identificando padrões e dificuldades comuns entre os estudantes.
- 2 Como pretende:** Utilizando ferramentas de captura de eventos durante as aulas de programação, processando os dados coletados e organizando-os em um repositório acessível.
- 3 Por que pretende:** Para melhorar o ensino de programação, oferecendo outras características, objetos de estudos baseados em dados quantitativos e criando um recurso (*dataset*) que possa ser utilizado por outros pesquisadores e educadores.

Tarefas metodológicas

- 1 Desenvolver um *plugin* para registro de eventos durante as aulas de programação.
- 2 Implementar o *plugin* na IDE utilizada na disciplina, com a anuência da instituição e dos estudantes.
- 3 Coletar, sincronizar e armazenar dados de eventos gerados durante o experimento.
- 4 Desenvolver um *dataset* com os dados coletados, organizados por turma, instituição e disciplina.

Contribuições esperadas

- 1 Um Mapeamento Sistemático da Literatura sobre uso de logs no ensino de programação focado principalmente nas instituições de ensino brasileiras.
- 2 Desenvolvimento de um plugin para uma IDE que possa auxiliar na coleta de logs durante o uso do programa e desenvolvimento de códigos.
- 3 Desenvolvimento e disponibilidade de um *dataset* dos logs gerados de forma aberta, respeitando a privacidade e dados sigilosos de todos os participantes.

Metodologia

Métodos de desenvolvimento

- Revisão Sistemática da Literatura
- Survey Exploratório
- Design Science Research

Revisão Sistemática da Literatura

Lista de repositórios

Repositório	Resultados
Scopus	23
IEEE Xplore	2
SOL SBC	2
ACM	127
WILEY	9

Tabela 4 – Lista de repositórios de pesquisa utilizados no estudo.

Palavras-chave e Operadores

Palavras-chave e Operadores
programming AND teaching AND log AND NOT (log-based) AND NOT (metrics) AND systematic AND review
programming AND teaching AND log AND NOT (log-based) AND (integrate AND development AND environment) AND NOT (visual AND programming) AND NOT (web AND application)
telemetry AND education AND programming AND log
ontology AND education AND programming AND log AND dataset
artefact AND cognitive AND learning
background AND teaching AND log AND NOT (log-based) AND (systematic AND review)
log AND ("teaching programming"OR "learning programming") AND ("IDE"OR "integrated development environment")) NOT ("log-based"OR "lms"OR "learning management system"
(log AND ("teaching programming"OR "learning programming") AND ("IDE"OR "integrated development environment")) NOT ("log-based"OR "lms"OR "learning management system"OR "game-based"OR "game based"OR "gamification")

Tabela 5 – Combinações de palavras-chave e operadores utilizados na pesquisa.

Critério de Inclusão

- 1 Relevância temática
- 2 Revisão sistemática

Critério de Exclusão

- 1 Log-based
- 2 Uso de logs em outras áreas da educação
- 3 Learning Management System(LMS)
- 4 Game-based learning

Resultados da busca e caracterização dos Estudos

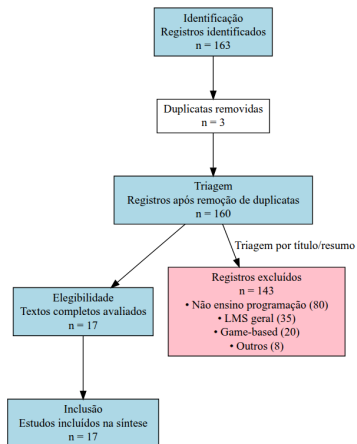


Figura: Fluxograma PRISMA do processo de seleção de estudos

Características dos 17 estudos incluídos na síntese qualitativa

Estudo	Ano	Foco Principal	Tipo de Log
Arabyarmohamady et al.	2012	Deteção de plágio	Estilo de codificação
Annamaa	2015	IDE educacional	Logs de sessão Python
Raabe et al.	2005	Dificuldades de aprendizagem	Ambiente educacional
Liu et al.	2018	Comportamento do usuário	Logs de execução
Umezawa et al.	2020	Análise de erros lógicos	Histórico de arquivos
Pereira et al.	2020	Comportamento estudantil	Logs de IDE + Online Judge
Gu et al.	2022	Práticas de logging	Revisão sistemática
Silva et al.	2022	Predição de dificuldade	Métricas de código
Mangaroska et al.	2022	Estados cognitivos	Logs Eclipse + multimodal
Da Silva	2022	Análise de comunidades	Comportamento online
Nguyen et al.	2023	Avaliação automática	Git logs + qualidade
Educomp	2023	Complexidade vs dificuldade	Métricas de exercícios
Gupta et al.	2024	Padrões de programação	Clickstream data
Arguson et al.	2022	Erros de compilação	Logs de compilação
Liz-Dominguez et al.	2022	Desempenho acadêmico	Logs de LMS
Yang et al.	2021	Uso de logs na indústria	Práticas profissionais
Cândido et al.	2021	Monitoramento baseado em logs	Revisão sistemática

Tabela 3 – Características dos 17 estudos incluídos na síntese qualitativa

Questões de Pesquisas

- 1 QP1 - Quais são os principais objetivos e aplicações do uso de logs no ensino de programação?
- 2 QP2 - Quais ferramentas, métodos e métricas são mais utilizados para capturar, armazenar e analisar logs em ambientes de ensino de programação?

Questão de Pesquisa 1

Objetivos categorizados em quatro áreas principais

Os objetivos identificados foram categorizados em quatro áreas principais:

- **Deteccção de Dificuldades** (6 estudos): Umezawa (2020), Pereira (2020), Silva (2022), Educomp (2023), Arguson (2022), Raabe (2005)
- **Análise de Comportamento** (5 estudos): Gupta (2024), Liu (2018), Mangaroska (2022), Da Silva (2022), Liz-Domínguez (2022)
- **Qualidade de Código** (3 estudos): Arabyarmohamady (2012), Nguyen (2023), Annamaa (2015)
- **Práticas e Ferramentas** (3 estudos): Gu (2022), Yang (2021), Cândido (2021)

3W1H

De acordo com (Gu *et al.*, 2022) “A obtenção de logs é um processo que deve ser guiado por quatro questões fundamentais, conhecidas como 3W1H:

- Why (Por quê): Qual é o objetivo da coleta de logs?
- Where (Onde): Em quais partes do sistema ou código os logs devem ser coletados?
- What (O que): Quais informações ou eventos devem ser registrados nos logs?
- How (Como): De que forma os logs serão coletados, armazenados e analisados?

Questão de Pesquisa 2

Ferramentas, métodos e métricas

- CodeBench (Pereira *et al.*, 2020)
- Alignment-based matching (Lit *et al.*, 2018)
- Plugin para Eclipse que possui uma aba - (Mangaroska *et al.*, 2022)
- Thonny (Annamaa, 2015)
- ProgEdu (Nguyen; Ha; Chen, 2023)

Survey

Para entender o uso de logs no ensino de programação, buscamos informações de outras instituições de ensino. Para ter informações sobre como alguns nós da Rede EPTC tratam o assunto, foi elaborado um questionário. A partir das respostas, os dados foram tabulados e gráficos foram gerados a partir dessas informações.



Figura: Link para o Google Forms

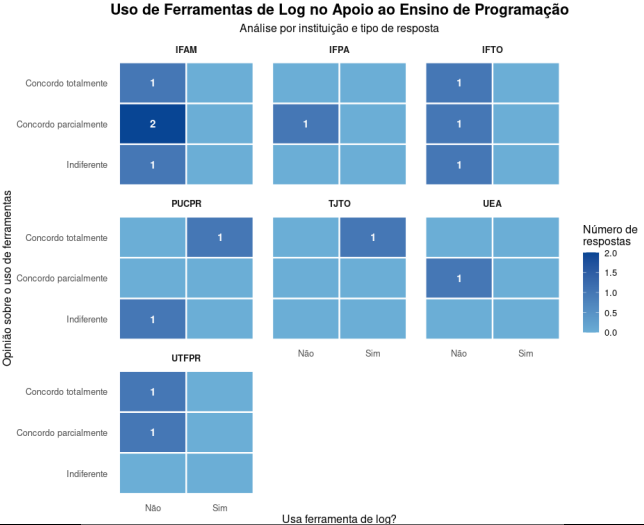
Instrumento de pesquisa

2. Você usa alguma ferramenta de log no apoio ao ensino de programação?
() Sim () Não
3. O uso de ferramentas de logs pode auxiliar no monitoramento e compreensão no empenho dos estudantes?
(1) Discordo Totalmente (2) Discordo Parcialmente (3) Neutro (4) Concordo Parcialmente (5) Concordo Totalmente
4. O uso de ferramentas de logs pode ajudar a tomar decisões para melhorar o rendimento e evitar a desistência de uma disciplina?
(1) Discordo Totalmente (2) Discordo Parcialmente (3) Neutro (4) Concordo Parcialmente (5) Concordo Totalmente
5. O uso de ferramentas de logs pode auxiliar no ensino de programação?
(1) Discordo Totalmente (2) Discordo Parcialmente (3) Neutro (4) Concordo Parcialmente (5) Concordo Totalmente

Continua

6. Um dataset pode ser criado e estudado a partir de logs no ensino da programação?
(1) Discordo Totalmente (2) Discordo Parcialmente (3) Neutro (4) Concordo Parcialmente (5) Concordo Totalmente
7. Sobre usar ferramentas de correções de códigos mais automatizado possível nas aulas de ensino de programação. O que pensa a respeito sobre o uso?
(1) Discordo Totalmente (2) Discordo Parcialmente (3) Neutro (4) Concordo Parcialmente (5) Concordo Totalmente
8. Plugins podem auxiliar na criação/ensino de algoritmos?
(1) Discordo Totalmente (2) Discordo Parcialmente (3) Neutro (4) Concordo Parcialmente (5) Concordo Totalmente
9. Qual IDE é utilizado nas aulas?
() Visual Studio () IntelliJ IDEA () Eclipse () Outro - Descrever
10. Ferramenta de análise de algoritmos
() Code Bench () Juiz Online () LAPLE () Outro: Descrever
11. Se tiver disponível um plugin de log para o apoio ao ensino de programação, você usaria?
() Sim () Não

Questões 2 e 3



Questões 4 e 5

Comparação das Opiniões sobre Uso de Ferramentas de Log

Cada quadrado representa uma resposta

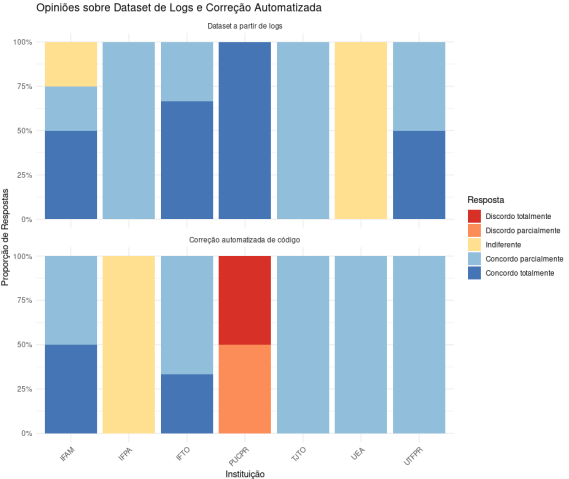
Auxílio no ensino

Tomada de decisões

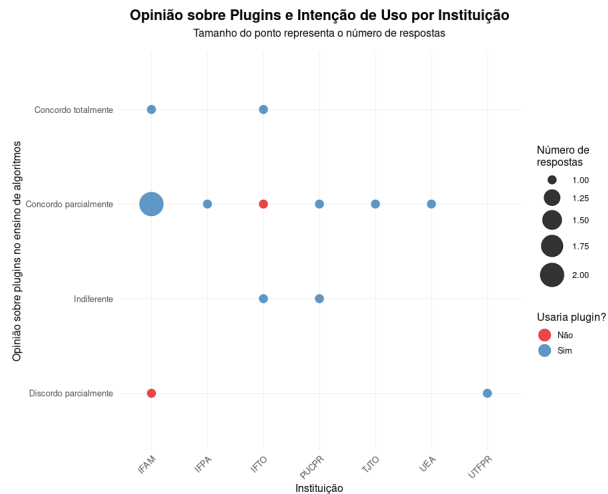


Resposta: ■ Discordo totalmente ■ Discordo parcialmente ■ Indiferente ■ Concordo parcialmente ■ Concordo totalmente

Questões 6 e 7



Questões 9 e 11



Fonte: Dados coletados da pesquisa

Design Science Research

Design Science Research

Este trabalho adota a abordagem de Design Science Research (DSR), conforme proposto por (*Peppers et al., 2007*) e (*Hevner et al., 2004*). O DSR é adequado para pesquisas que visam desenvolver e avaliar artefatos que resolvam problemas identificados em contextos organizacionais e educacionais.

O processo de DSR será executado em quatro atividades principais:

- 1 Identificação do Problema e motivação
- 2 Definição dos Objetivos da solução
- 3 Design e Desenvolvimento
- 4 Especificação de Requisitos do Artefato

Identificação do Problema e Motivação

O problema central identificado é a carência de ferramentas específicas para captura e análise de logs acadêmicos no ensino de programação, especialmente no contexto da região Amazônica. Esta lacuna limita a capacidade de docentes e instituições em identificar dificuldades de aprendizagem de forma objetiva e baseada em dados.

Definição dos Objetivos da Solução

Com base no problema identificado, definiram-se os seguintes objetivos:

Objetivo Geral: Desenvolver e validar um *plugin* para coleta e análise de logs durante o processo de ensino-aprendizagem de programação.

Objetivos Específicos:

- 1 Desenvolver um *plugin* para registro de eventos em IDE
- 2 Implementar o plugin na IDE utilizada na disciplina
- 3 Coletar, sincronizar e armazenar dados de eventos
- 4 Desenvolver um dataset com os dados coletados
- 5 Validar a abordagem por meio de experimentos controlados

Design e Desenvolvimento

O design e desenvolvimento segue uma abordagem iterativa, pois os dados serão coletados quando os estudantes desenvolvem as atividades envolvendo algoritmos. A arquitetura do artefato compreendendo quatro módulos principais:

- Módulo de captura: Implementa a coleta de palavras reservadas e o carimbo temporal (*timestamp*)
- Módulo de serialização: Transforma os eventos em uma estrutura JSON
- Módulo de armazenamento: Gerencia os arquivos localmente
- Módulo de configuração: Confirma se a extensão está funcionando corretamente e coletando os logs

Especificação de requisito do artefato

Para garantir que o *plugin Logado* atenda aos objetivos da pesquisa e às necessidades dos usuários finais, foi realizada uma etapa de especificação de requisitos. A análise inicial identificou dois atores principais, cujas necessidades são sumarizadas na Tabela 2.

Tabela: Análise de atores e necessidades do sistema

Ação	Discente	Pesquisador
Funções necessárias	Enviar dados de execução	Desenvolver/testar plugin
Necessidade principal	Instalar e usar o plugin	Validar processamento de logs
Operações CRUD	Enviar (criar) logs	Gerenciar usuários e data-set

Caso de uso: Discentes

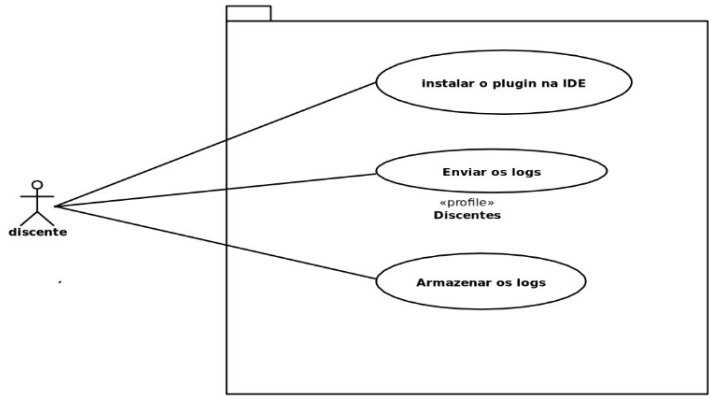


Figura: Visão do Estudante

Caso de uso: Pesquisa

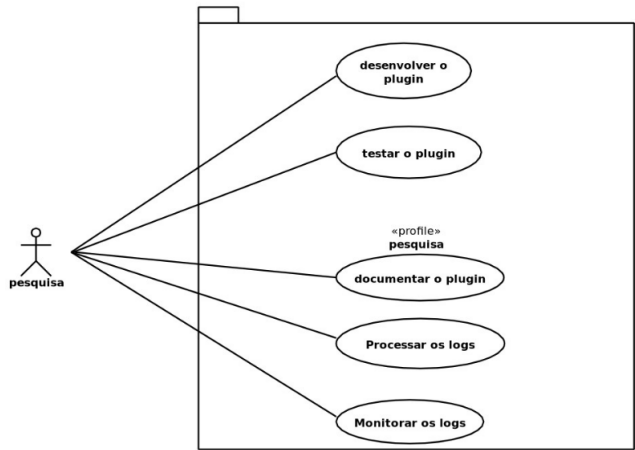


Figura: Visão da Pesquisa

Requisitos Funcionais - Discente

Os requisitos funcionais definem as funcionalidades que o sistema deve executar.

Para o ator **Discente**, um *plugin* deve:

- **RF01:** Registrar automaticamente eventos de edição de código (e.g., inserção, deleção de caracteres).
- **RF02:** Capturar eventos de compilação e execução de código, incluindo sucesso ou falha.
- **RF03:** Armazenar localmente os logs com *timestamp* preciso.
- **RF04:** Transmitir os logs anonimizados para um servidor remoto de forma assíncrona.
- **RF05:** Exibir notificações de confirmação de operações para o usuário.

Para o ator **Pesquisador**, um *plugin* deve:

- **RF06:** Fornecer uma interface de configuração para parametrizar os eventos a serem capturados.
- **RF07:** Gerar identificadores únicos anônimos para cada sessão de uso.

Requisitos Não-funcionais

Estes requisitos definem os critérios de qualidade para o sistema:

- **RNF01 (Desempenho):** um *plugin* não deve impactar significativamente o desempenho da IDE. O tempo de resposta deve ser inferior a 100ms para operações de registro.
- **RNF02 (Usabilidade):** A instalação e o uso devem ser simples, não requerendo configuração complexa por parte do discente.
- **RNF03 (Confiabilidade):** um *plugin* deve garantir a persistência dos logs mesmo em caso de fechamento inesperado da IDE.
- **RNF04 (Segurança e Privacidade):** Todos os dados pessoais devem ser anonimizados antes do envio ao servidor, conforme a LGPD.

Desenvolvimento do Artefado

LOGADO - Logs Acadêmico



Visual Studio Code



Logado

LOGADO

- Palavras reservadas (linguagem de programação)
- Carimbos temporais (timestamps)
- Salvar os logs (e armazenar, sincronizar, agrupar)

Exemplo de algoritmo

```
1 let numero_paginas = parseInt(prompt("Número de páginas"))
2 let tipo_impressao = prompt("Tipo de Impressão\n0 - Frente e verso\n1 - Frente")
3
4
5 let valor = 0
6
7 if(tipo_impressao=='0'){
8     valor = numero_paginas * 0.10
9     document.writeln('Páginas impressas: ${numero_paginas}<br> Valor final: R$ ${valor.toFixed(2)}')
10 }
11 else{
12     valor = numero_paginas * 0.15
13     document.writeln('Páginas impressas: ${numero_paginas}<br> Valor final: R$ ${valor.toFixed(2)}')
14 }
15
```

Listing 2.1 – Algoritmo de cálculo de impressão - exemplo didático que simula o desenvolvimento típico de um estudante iniciante

Exemplo de rastreamento

```

1  [
2  {
3    "keyword": "let",
4    "timestamp": "2025-12-11T10:15:30.250Z",
5    "file": "impressao.js",
6    "line": 1,
7    "column": 1
8  },
9  {
10   "keyword": "parseInt",
11   "timestamp": "2025-12-11T10:15:33.500Z",
12   "file": "impressao.js",
13   "line": 1,
14   "column": 20
15 },
16 {
17   "keyword": "prompt",
18   "timestamp": "2025-12-11T10:15:33.520Z",
19   "file": "impressao.js",
20   "line": 1,
21   "column": 30
22 },
23 {
24   "keyword": "let",
25   "timestamp": "2025-12-11T10:15:37.100Z",
26   "file": "impressao.js",
27   "line": 2,
28   "column": 1
29 },
30 {
31   "keyword": "prompt",
32   "timestamp": "2025-12-11T10:15:40.300Z",
33   "file": "impressao.js",
34   "line": 2,
35   "column": 21
36 },
37 {
38   "keyword": "let",
39   "timestamp": "2025-12-11T10:15:44.800Z",
40   "file": "impressao.js",
41   "line": 4,
42   "column": 1
43 },
44 ]

```

Continuação

```
--
44  {
45    "keyword": "if",
46    "timestamp": "2025-12-11T10:15:48.150Z",
47    "file": "impressao.js",
48    "line": 6,
49    "column": 1
50  },
51  {
52    "keyword": "document",
53    "timestamp": "2025-12-11T10:15:52.400Z",
54    "file": "impressao.js",
55    "line": 8,
56    "column": 5
57  },
58  {
59    "keyword": "writeln",
60    "timestamp": "2025-12-11T10:15:52.420Z",
61    "file": "impressao.js",
62    "line": 8,
63    "column": 14
64  },
65  {
66    "keyword": "else",
67    "timestamp": "2025-12-11T10:15:56.700Z",
68    "file": "impressao.js",
69    "line": 11,
70    "column": 1
71  },
72  {
73    "keyword": "document",
74    "timestamp": "2025-12-11T10:16:01.200Z",
75    "file": "impressao.js",
76    "line": 13,
77    "column": 5
78  },
79  {
80    "keyword": "writeln",
81    "timestamp": "2025-12-11T10:16:01.220Z",
82    "file": "impressao.js",
83    "line": 13,
84    "column": 14
85  }
86  ]
```


Visual Studio Code Extension

[Version 1.107](#) is now available! Read about the new features and fixes from November.

Overview

GET STARTED

Your First Extension

Extension Anatomy

Wrapping Up

EXTENSION CAPABILITIES

EXTENSION GUIDES

UX GUIDELINES

LANGUAGE EXTENSIONS

TESTING AND PUBLISHING

ADVANCED TOPICS

REFERENCES

Your First Extension

In this topic, we'll teach you the fundamental concepts for building extensions. Make sure you have [Node.js](#) and [Git](#) installed.

First, use [Yeoman](#) and [VS Code Extension Generator](#) to scaffold a TypeScript or JavaScript project ready for development.

- If you do not want to install Yeoman for later use, run the following command:

Bash

```
npx --package yo --package generator-code -- yo code
```

- If you instead want to install Yeoman globally to ease running it repeatedly, run the following command:

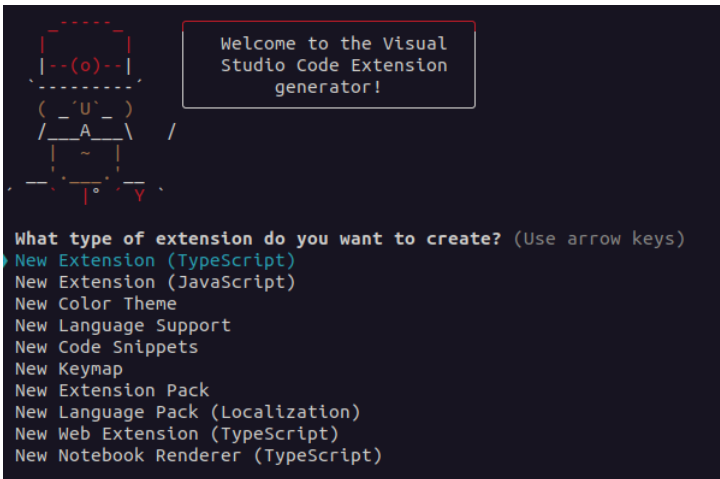
Bash

```
npm install --global yo generator-code  
  
yo code
```

ON THIS PAGE

- Developing the extension
- Debugging the extension
- Next steps

yo-code



Cronograma

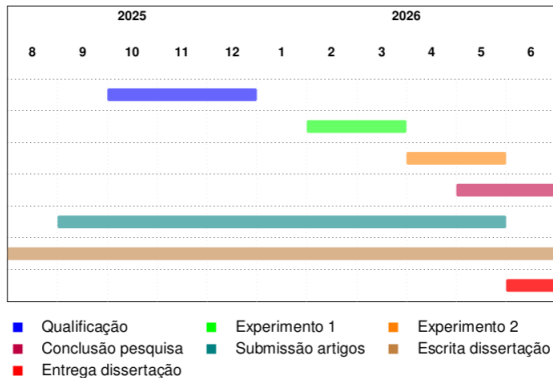


Figura 3 – Cronograma do trabalho de mestrado

Artigo - Evento

LACLO



Referências

ANNAMAA, A. Thonny, a python ide for learning programming. *In*: PROCEEDINGS OF THE 2015 ACM CONFERENCE ON INNOVATION AND TECHNOLOGY IN COMPUTER SCIENCE EDUCATION. 2015. **Anais [...]** [S.l.: s.n.], 2015. p. 343–343.

ARABYARMOHAMADY, S.; MORADI, H.; ASADPOUR, M. A coding style-based plagiarism detection. *In*: IEEE. PROCEEDINGS OF 2012 INTERNATIONAL CONFERENCE ON INTERACTIVE MOBILE AND COMPUTER AIDED LEARNING (IMCL). 2012. **Anais [...]** [S.l.], 2012. p. 180–186.

CLEMENTE, R. G. **Uma arquitetura para processamento de eventos de log em tempo real**. 2008. 15–77 p. Dissertação (Mestrado) — Pontifícia Universidade Católica do Rio de Janeiro 2008.

DU, H. S.; WAGNER, C. Learning with weblogs: An empirical investigation. *In*: IEEE. PROCEEDINGS OF THE 38TH ANNUAL HAWAII INTERNATIONAL CONFERENCE ON SYSTEM SCIENCES. 2005. **Anais [...]** [S.l.], 2005. p. 7b–7b.

FU, Q. *et al.* Where do developers log? an empirical study on logging practices in industry. *In*: COMPANION PROCEEDINGS OF THE 36TH INTERNATIONAL CONFERENCE ON SOFTWARE ENGINEERING. 2014, Hyderabad India. **Anais [...]** Hyderabad India: ACM, 2014. p. 24–33. ISBN 978-1-4503-2768-8. Disponível em: <https://dl.acm.org/doi/10.1145/2591062>.

Referências

KENNEDY, A. R. **Towards a data-driven analysis of programming tutorials' telemetry to improve the educational experience in introductory programming courses.** 2015. Tese (Doutorado) 2015.

KITCHENHAM, B. Procedures for performing systematic reviews. **Keele, UK, Keele University**, v. 33, n. 2004, p. 1–26, 2004.

Referências

- LIMA, N. A. *et al.* **Avaliação da aprendizagem nos Institutos Federais em Goiás durante a pandemia: uma investigação documental.** [S.l.], 2022.
- LIU, C. *et al.* User behavior discovery from low-level software execution log. **IEEJ Transactions on Electrical and Electronic Engineering**, Wiley Online Library v. 13, n. 11, p. 1624–1632, 2018.
- MANGAROSKA, K. *et al.* Exploring students' cognitive and affective states during problem solving through multimodal data: Lessons learned from a programming activity. **Computer Assisted Learning**, v. 38, n. 1, p. 40–59, fev. 2022. ISSN 0266-4909, 1365-2729. Disponível em: <https://onlinelibrary.wiley.com/doi/10.1111/jcal.12590>.
- NGUYEN, B.-A.; HA, T.-V. T.; CHEN, H.-M. Analyzing git log in an code-quality aware automated programming assessment system: A case study. **TVU Journal of Science**, v. 13, n. 3, p. 1–15, 2023. ISSN 2815-6099. Disponível em: <https://journal.tvu.edu.vn/index.php/journal/article/view/2430>.



Referências

- PAGE, M. J. *et al.* The prisma 2020 statement: an updated guideline for reporting systematic reviews. **Systematic reviews**, Springer v. 10, n. 1, p. 1–11, 2021.
- PEFFERS, K. *et al.* A design science research methodology for information systems research. **Journal of management information systems**, Taylor & Francis v. 24, n. 3, p. 45–77, 2007.
- PEREIRA, F. D. *et al.* Using learning analytics in the amazonas: understanding students' behaviour in introductory programming. **British journal of educational technology**, Wiley Online Library v. 51, n. 4, p. 955–972, 2020.
- PRESSMAN, R. S. **Engenharia de Software: Uma Abordagem Profissional**. 7. ed. [S.l.]: AMGH, 2014.
- QIAN, Y.; LEHMAN, J. Students' misconceptions and other difficulties in introductory programming: A literature review. **ACM Transactions on Computing Education (TOCE)**, ACM New York, NY, USA v. 18, n. 1, p. 1–24, 2017.
- RAABE, A. L. A.; SILVA, J. d. Um ambiente para atendimento as dificuldades de aprendizagem de algoritmos. *In*: SN. XIII WORKSHOP DE EDUCAÇÃO EM COMPUTAÇÃO (WEI'2005). SÃO

Referências

RICE, R. E.; BORGMAN, C. L. The use of computer-monitored data in information science and communication research. **Journal of the American Society for Information Science**, Wiley Online Library v. 34, n. 4, p. 247–256, 1983.

RODRIGUES, F. C. R.; GAVA, R. Universidades federais no estado de minas gerais: Um estudo comparativo. **REAd | Porto Alegre – Edição 83**, SciELO Brasil,, 2016.

SILVA, E. S. *et al.* Previsão de indicadores de dificuldade de questões de programação a partir de métricas do código de solução. *In*: SBC. ANAIS DO XXXIII SIMPÓSIO BRASILEIRO DE INFORMÁTICA NA EDUCAÇÃO. 2022. **Anais [...]** [S.l.], 2022. p. 859–870.

SOMMERVILLE, I. **Engenharia de Software**. 9. ed. [S.l.]: Pearson, 2011.

UMEZAWA, K. *et al.* Analysis of logic errors utilizing a large amount of file history during programming learning. *In*: IEEE. 2020 IEEE INTERNATIONAL CONFERENCE ON TEACHING, ASSESSMENT, AND LEARNING FOR ENGINEERING (TALE). 2020. **Anais [...]** [S.l.], 2020. p. 630–634.

VALENTE, M. T. **Engenharia de Software Moderna**. 1. ed. [S.l.]: Independente, 2020.

Agradecimentos

Obrigado!

